

Teaching a Fracture Classifier Where to Look

CNN vs ScatNet, six XAI methods and a joint classifier–detector on FracAtlas

Silver Gjeka (VR527645) Sevastian Guzun (VR508193)

Department of Computer Science
Master's Degree in Artificial Intelligence
University of Verona

Visual Intelligence

Data: FracAtlas (Abdedeen et al., *Scientific Data*, 2023)
Code: github.com/silvergjeka22/bone-fracture-detection

Outline

Part I – Classify and explain

1. The problem and the data
2. CNN vs ScatNet (+ ResNet18)
3. Six XAI methods: faithful? on the fracture?

Part II – Teach it where to look

4. Three models: A, B and C, and their losses

Part III – Results

5. Do the explanations find the fracture?
6. Test X-rays
7. Every model side by side
8. Can our model help YOLO?
9. Limits and conclusion

Part I

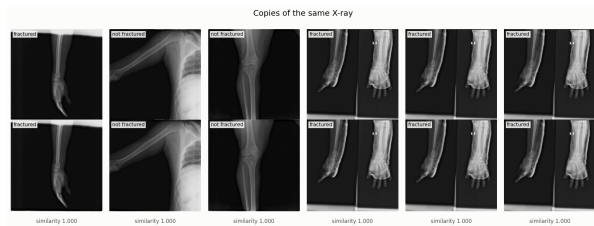
Classify and Explain

Which model sees fractures, and where does it look?

The Problem: Right, but for the Right Reason?

- A model can be **accurate for the wrong reasons**: a metal plate, a cast or the body part can predict “fractured” better than a thin fracture line.
- **XAI** draws a heatmap of the evidence. It helps only if it is **faithful** (shows what the model uses) **and** the model uses the **fracture**.
- FracAtlas has the **radiologists’ boxes**: we can **measure** where a model looks.
- Our question: can we **teach** a classifier to look at the fracture, without losing accuracy?

The Data: FracAtlas, Without Leakage

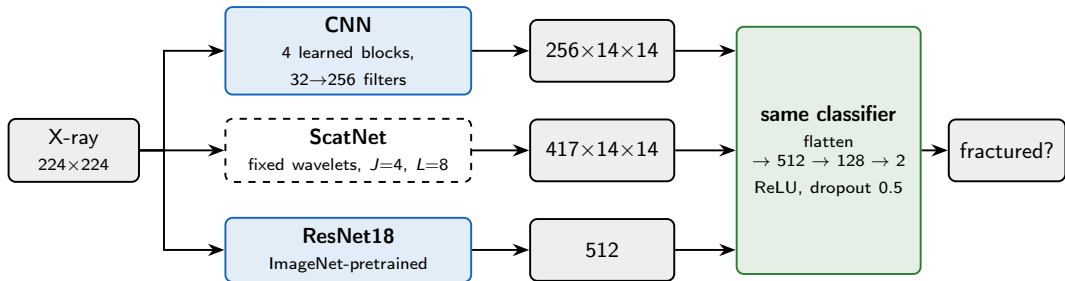


Copies of the same X-ray (flipped, resized, brighter) found automatically.

- 4,083 X-rays; **717 fractured** (1 in 6), 922 boxes.
- Grey, 224×224 (448 in our method), values in $[-1, 1]$.
- **8.6%** of the X-rays have a copy (158 groups): a split **keeps every group together**.
- Split $\approx 70/15/15$: **2,920** train, **577** val, **586** test (104 fractured).

Always saying “not fractured” already scores **82.3%**: look at **F1** and **fractures found**.

Three Feature Extractors, One Classifier



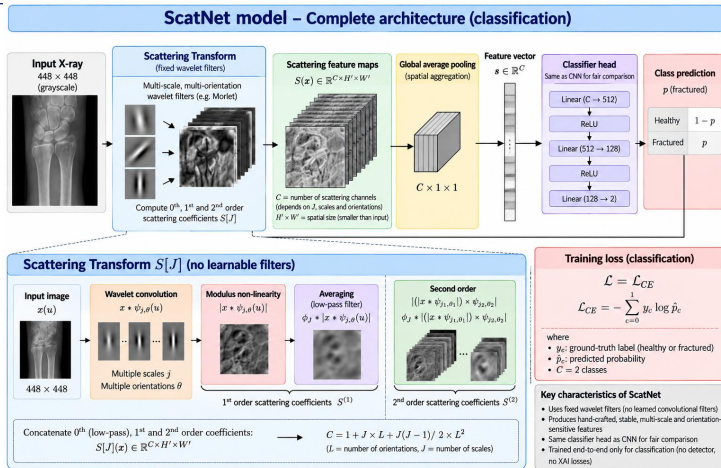
CNN: every filter **learned** from our 2,920 X-rays (first 7×7).

ScatNet: **nothing learned** before the classifier; stable to small deformations.

ResNet18: filters learned on **1.2M ImageNet** photos, then fine-tuned.

Only the classifier's **input size** differs (50,176 / 81,732 / 512). Weighted loss (1 in 6 fractured), 20 epochs, 5-fold grouped CV.

ScatNet: The Complete Architecture



In our run: input 224×224, $J=4$ scales, $L=8$ directions $\Rightarrow C = 1 + 32 + 384 = 417$ maps of 14×14, then log + BatchNorm, flattened into the same classifier (Bruna & Mallat, 2013; Kymatio).

CNN vs ScatNet (+ ResNet18): Results

	CNN	ScatNet	ResNet18
CV accuracy	73.0±9.0	77.5±2.9	89.0±1.0
CV F1	35.7±13.0	38.0±2.6	66.6±2.7
test accuracy	80.9	80.0	91.5
test F1	23.3	31.6	73.7
fractures found	16%	26%	67%
AUC	68.6	72.7	91.2
parameters	26.1M	41.9M	11.5M

McNemar: CNN $\approx$ ScatNet ($p = 0.55$); ResNet18 beats both ($p < 0.001$).

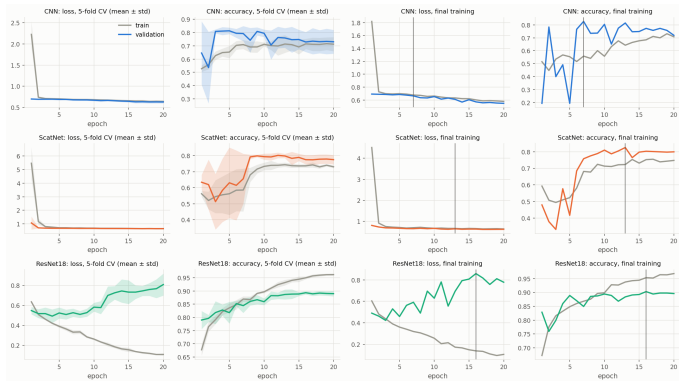
Why CNN and ScatNet fail

- Both **underfit** (loss stuck at 0.6) and stay **below** the 82.3% of “never fractured”.
- Only 511 fractured X-rays; a fracture is 1–2 px wide at 224 px.
- Flattening a 14×14 grid: 26–42M parameters in the classifier.

Why ResNet18 wins

- Pretrained edge and texture detectors that 2,920 X-rays cannot teach.
- Its val loss rises after ≈ 10 epochs; we keep the epoch with the best val accuracy (16).

Learning Curves



Grey: train; colour: validation. Columns 1–2: 5-fold CV (mean $\pm$ std); columns 3–4: final training (vertical line = epoch kept).

CNN (top) and ScatNet (middle)

- Train and val loss flatten at ≈ 0.6 – 0.7 after 2 epochs: **underfitting**, not overfitting.
- Accuracy jumps between epochs: the model flips between “all healthy” and guessing.

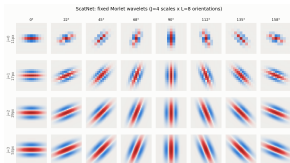
ResNet18 (bottom)

- Train loss keeps falling, val loss **rises after ≈ 10 epochs: overfitting.**
- Val accuracy still climbs slowly: we keep the epoch with the **best val accuracy** (16), not the last.

First-Layer Filters: What Each Model Uses



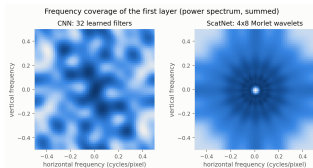
CNN: 32 filters 7×7 learned from scratch: **noisy**, no clear edges.



ScatNet: fixed Morlet wavelets, **every** scale and direction.



ResNet18: ImageNet filters: clean edges, bars and blobs.



- In frequency: the CNN covers the spectrum **unevenly** (2,920 X-rays are too few to learn clean filters); ScatNet covers it **evenly** by construction.
- But clean filters are not enough: ScatNet still fails, because it lacks a **learned** hierarchy above them.
- ResNet18 has both: clean filters **and** 16 more learned conv layers above them.

Six XAI Methods: How Each One Works

method	family	how it works	good / bad	time (ms)
Saliency	gradient	how much the score changes if one pixel changes a little	fast / noisy	15
Integrated Gradients	gradient	the same, averaged on the way from a black image to the X-ray	complete / slower	194
Guided Backprop	gradient	the gradient, keeping only the positive signals	sharp / draws every edge	10
Grad-CAM	gradient	which regions of the last feature grid push towards "fractured"	per class / coarse; not on ScatNet	5
Occlusion	masking	cover a 16×16 square with black, see how much the score drops	honest / slow	767
LIME	masking	hide random squares many times, fit a simple linear model	honest / slowest	1,377

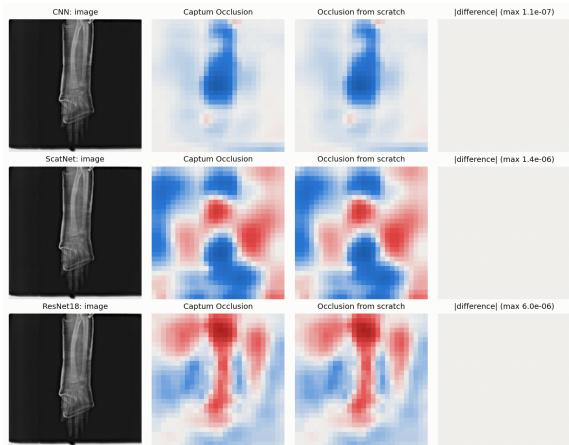
time = milliseconds to explain one X-ray (ResNet18, Kaggle T4 GPU)

- **Gradient methods:** one pass back through the network $\Rightarrow$ **fast** (5–194 ms) and pixel-sharp. **But** they only *estimate* what a tiny change would do: noisy, and in a deep network they can show pixels the model does not really use.
- **Masking methods:** hide parts of the X-ray and run the model again, hundreds of times $\Rightarrow$ **slow** (0.8–1.4 s). **But** they *measure* the real effect of removing a region: more faithful, though coarser (16×16 squares).

Occlusion, Written From Scratch

Given: model f , X-ray x , class c , window $w=16$,
stride $s=8$
 $z \leftarrow f(x)_c$; $A \leftarrow 0$; $N \leftarrow 0$
for every window position (i, j) , in batches **do**
 $\hat{x} \leftarrow x$ with the window set to black
 $A[\text{window}] += z - f(\hat{x})_c$
 $N[\text{window}] += 1$
end for
return A/N ▷ mean drop per pixel

- Plain PyTorch, all positions scored in batches.
- **Checked against Captum** (the standard library): on all three models the two maps are **the same**. Correlation $r = 1.000$; largest difference 0.0000086, only rounding. The difference maps on the right are empty.

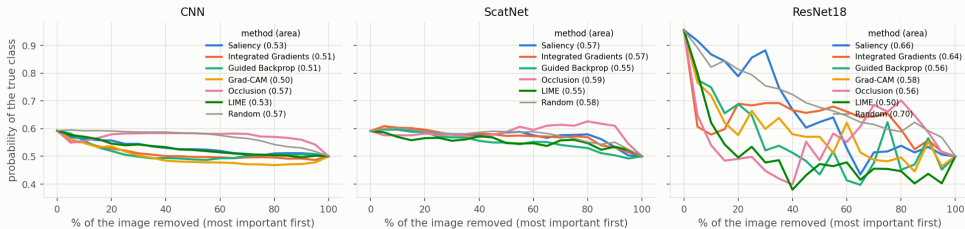


Our Occlusion, Captum's, and their difference.

Is the Map Faithful? The Deletion Test

We remove the parts the map calls **most important first** (16×16 squares) and watch the model's confidence. **It falls fast** $\Rightarrow$ the map is right (**low area**). Grey = random order.

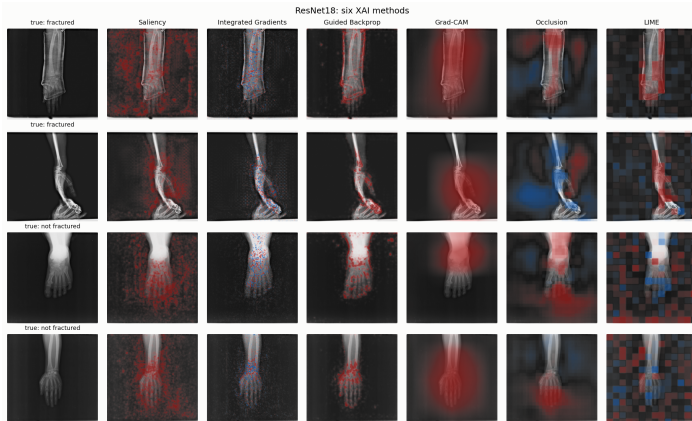
Deletion test: lower = more faithful attribution



- **ResNet18** starts very sure (0.95), so the drop is clear: **LIME** falls fastest (0.50), then Occlusion and Guided Backprop (0.56). Saliency and IG stay close to random (0.64–0.66 vs 0.70): their gradients saturate in a deep network.
- **CNN and ScatNet underfit:** they start at only ≈ 0.6 , close to a guess, so there is little to lose and the curves stay flat. Grad-CAM is still the best for the CNN (0.50); for ScatNet every method is close to random.

$\Rightarrow$ The best method **depends on the model**.

The Six Methods on Test X-rays (ResNet18)

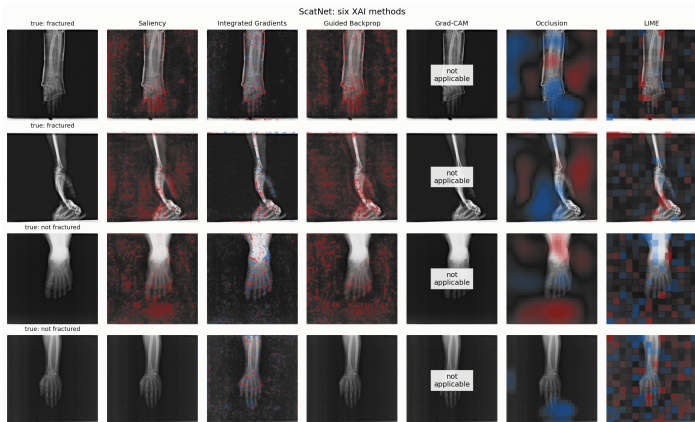


Rows 1–2: **fractured**; rows 3–4: **not fractured** (all classified correctly). Red = evidence **for** the answer, blue = **against**.

- **Saliency**: dots on the arm, but also on the black background: noisy.
- **Integrated Gradients**: fine red/blue speckles along the bones.
- **Guided Backprop**: clean bone outlines, like an edge detector.
- **Grad-CAM**: one big blob over the arm or foot: *this area*, not exactly where.
- **Occlusion**: large regions, mostly on the bone.
- **LIME**: red squares along the arm, blocky.

In short: all methods focus on the **bones**, each in its own way; none points at one small spot.

The Six Methods on Test X-rays (ScatNet)

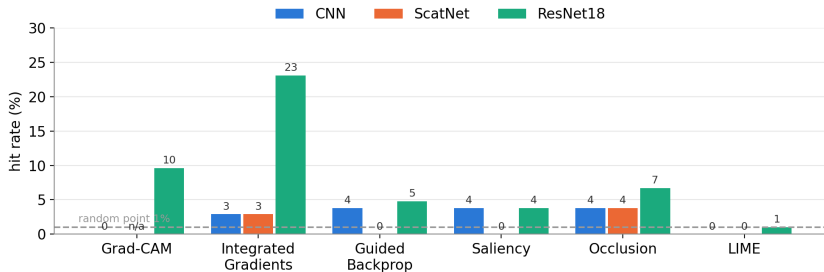


The same X-rays: rows 1–2 **fractured**, rows 3–4 **not fractured**.

- **Grad-CAM**: not applicable (no learned layer).
- **Saliency, Guided Backprop**: dots over the **whole image**, background included; almost empty on the healthy hand (row 4).
- **Integrated Gradients**: red/blue along the bone edges: the most structured map for ScatNet.
- **Occlusion**: big blobs, often on the **background** (under the foot, row 3).
- **LIME**: a checkerboard of red and blue squares, almost random.

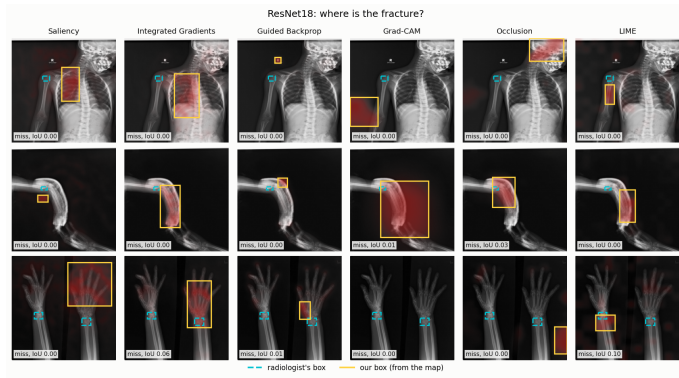
Compared with ResNet18: ResNet18 looks at the bones; ScatNet's maps are scattered over the whole image, as in the deletion test (close to random).

Does the Map Point at the Fracture?



Rarely. 104 fractured test X-rays; a random point hits 1%. The best is Integrated Gradients on ResNet18: **23%**. CNN and ScatNet stay at 0–4% with every method.

Why So Rarely? ResNet18's Six Maps



Three fractured test X-rays. Dashed: the radiologist's box; yellow: the box we cut from each map.

- The boxes are **tiny** (1% of the image); fractures are 1–2 px at 224 px.
- A third of the fractures are **missed** by the model.
- The model uses **shortcuts**: the chest, the elbow, the hand; the faithful maps honestly show them.
- **Grad-CAM is too blurry**: it sees the X-ray as only 7×7 big squares (32 px each), so it cannot point at a thin fracture line.

⇒ Faithful maps are not enough: the **model** must be taught to look at the fracture.

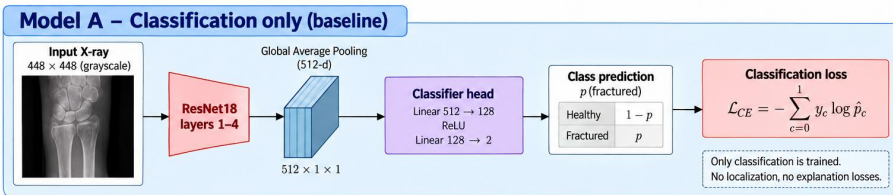
Part II

Teach the Model Where to Look

One network that classifies, detects, and is trained on its own explanation

Model A: Classifier Only (Baseline)

A, B and C: same network, same data, same training; only the loss changes.



What we did: our Part I ResNet18, trained again with three changes:

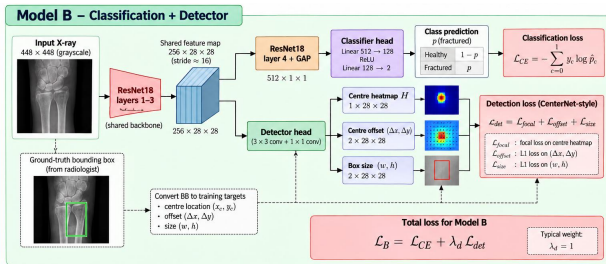
- **Bigger X-rays:** 448 px instead of 224, so thin fractures stay visible.
- **Balanced batches:** 16 X-rays, half fractured.
- **Finer Grad-CAM:** a 28×28 grid instead of 7×7 .

Loss: only cross-entropy (the label). Boxes come from a separate YOLO.

Why it falls short: nothing in its loss says **where**.

- Grad-CAM on the fracture: **35%**.
- Blur the fracture: $p(\text{fractured})$ 81% $\rightarrow$ **84%**: it does not need the fracture.

Model B: Classifier + Detector



How it works

1. A small **detector head** reads the **same 28x28 grid** as the classifier.
2. For each cell it predicts: a **centre heatmap** (fracture centre here?), an **offset** (exact spot inside the 16-px cell) and the **box size** (w, h).
3. The radiologist's box gives the targets; the losses compare.

Losses (N = fracture centres, $\hat{\cdot}$ = predicted)

$$\mathcal{L}_{focal} = -\frac{1}{N} \sum \begin{cases} (1 - \hat{H})^2 \log \hat{H} & \text{at a centre} \\ (1 - H)^4 \hat{H}^2 \log(1 - \hat{H}) & \text{elsewhere} \end{cases}$$

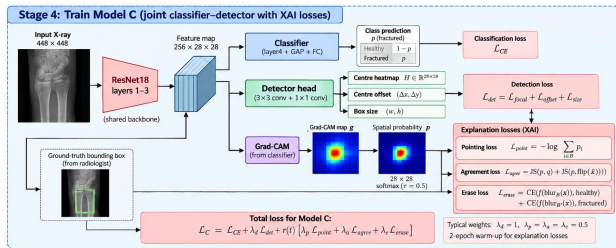
$$\mathcal{L}_{offset} = \frac{1}{N} \sum |\hat{o} - o| \quad \text{exact spot in the cell}$$

$$\mathcal{L}_{size} = \frac{1}{N} \sum |\hat{s} - s| \quad \text{box width and height}$$

Why it helps: both heads share the grid, so the detector teaches it **where** fractures are: Grad-CAM on the fracture 35% → **56%**.

Limit: the classifier is *not forced* to use that information. Blur the fracture and its answer falls only from 77% to 48%: half of its decision still comes from elsewhere. C fixes this.

Model C: Our Joint Model (B + Explanation Losses)



How it works: C is model B with one new idea: during training, the model also checks its own explanation.

At every step we compute its Grad-CAM, which shows **where the classifier looks** (p). We then check three things:

- does it look **inside the radiologist's box**?
- does it look **where the detector looks** (q), and at the same place on the **mirrored X-ray**?
- if we **blur the fracture**, does the answer become "healthy"?

Each "no" gives a loss that corrects the model.

Three extra losses (weight 0.5, after a 2-epoch warm-up)

$$\mathcal{L}_{point} = -\log \sum_{i \in \text{box}} p_i$$

uses the **whole Grad-CAM map** and pushes it **inside the box**

$$\mathcal{L}_{agree} = \text{JS}(p, q) + \text{JS}(p, \text{flip}(p_m))$$

must match the **detector's map** and the **mirror image**

JS = Jensen-Shannon divergence (0 = same map)

$$\mathcal{L}_{erase} = \text{CE}(\tilde{x}_{\text{fracture}}, \text{healthy}) + \text{CE}(\tilde{x}_{\text{patch}}, \text{fractured})$$

blur the fracture $\Rightarrow$ **healthy**; blur elsewhere $\Rightarrow$ still fractured

Why it helps: the model is trained on its **own explanation**, so it learns to look at the fracture: Grad-CAM on the fracture 35% $\rightarrow$ **67%**, with the same accuracy (93%).

Part III

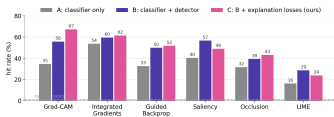
Results

A vs B vs C, test X-rays, and YOLO

A vs B vs C: Where They Look, and Why

Three questions on the 104 fractured test X-rays.

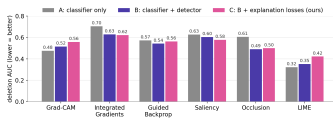
1. Does it look at the fracture?



Hit rate (higher = better).

- Grad-CAM: A 35% → B 56% → C 67%.
- All six methods improve from A to C.

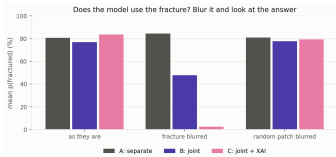
2. Is the map still faithful?



Deletion test (lower = better).

- About the **same** for A, B, C (0.55 / 0.52 / 0.54).
- All far better than random (0.70).

3. Does it need the fracture?

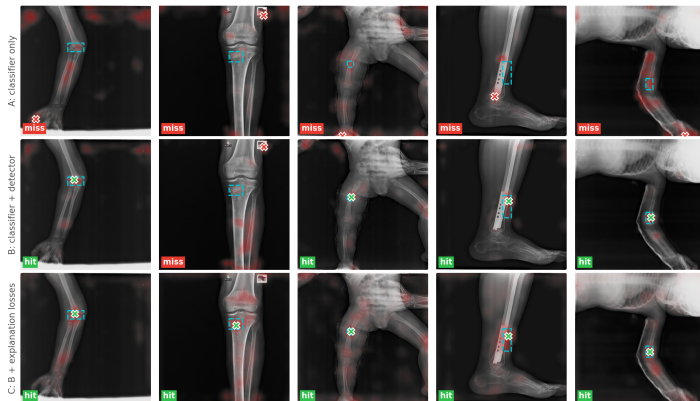


$p(\text{fractured})$, fracture blurred:

- A: 81% → 84% (**ignores it**)
- B: 77% → 48%
- C: 84% → **2.5% (needs it)**

Beginning vs now: at the beginning (Part I ResNet18) the best map found the fracture **23%** of the time; now (C) it is **67%**. The maps are **still faithful**: deletion 0.58 at the beginning, 0.54 now (random 0.70).

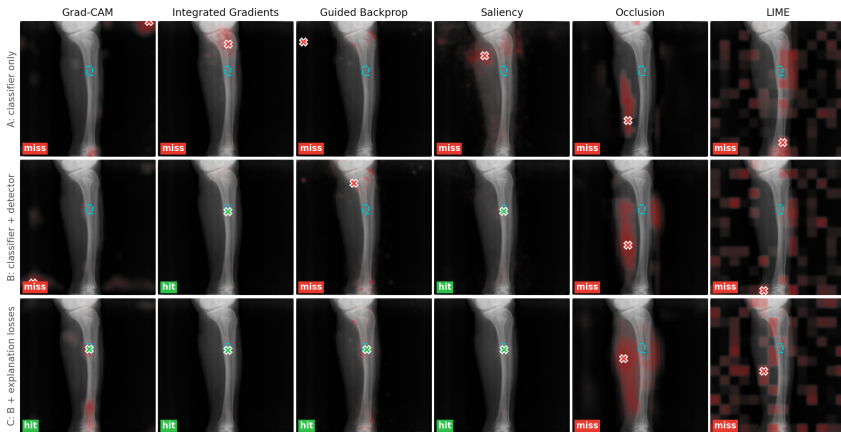
Test



Grad-CAM of A, B, C on five **new** fractured test X-rays.

- Red: where the model looks; **X**: its hottest point; dashed: the radiologist's box.
- **A** looks at a label, the hand, the foot, an image corner.
- **B** already finds 4 of 5; **C** finds all 5.
- **C** is one of **35** such X-rays; the opposite (**A** hits, **C** misses) happens **once**.

Test: One X-ray, Six Methods, Three Models



A thin lower-leg fracture. **A:** all six methods miss. **B:** IG and Saliency find it. **C:** four of six, including gradient methods outside its loss. Occlusion and LIME (32-px patches) stay too coarse for a 1-cell box.

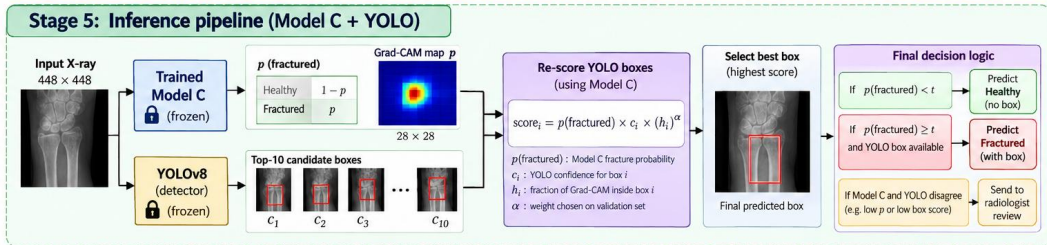
Every Model Side by Side

	what it learns	acc.	F1	best hit	blur drop	good at
CNN	filters from 2,920 X-rays	80.9	23.3	4%	–	– (underfits)
ScatNet	only the classifier	80.0	31.6	4%	–	stable features, no fractures
ResNet18	fine-tunes ImageNet	91.5	73.7	23%	–	classification
A: classifier only	same, 4× the pixels	93.3	81.0	54%	–4	finding fractures
B: + detector	+ where the centre is	94.2	82.3	60%	29	fewest false alarms
C: ours	+ where it looks, blur test	93.0	80.4	67%	81	looking at the fracture
YOLOv8	only boxes	–	–	79% (box)	–	placing the box

“best hit”: best of the six maps (for YOLO: its most confident box). “blur drop”: points of $p(\text{fractured})$ lost when the fracture is blurred. Same 586 test X-rays for all.

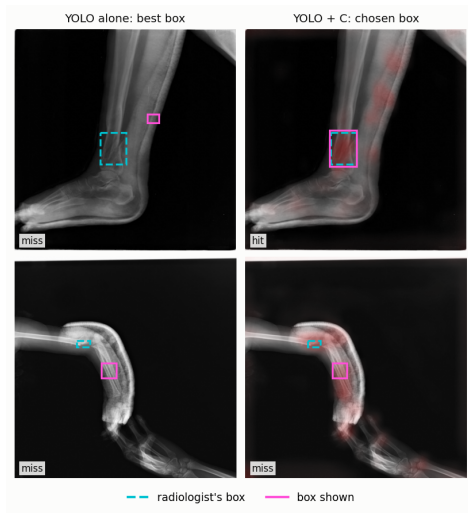
A, B and C are within each other's confidence intervals (± 2 points; a second run gave 91.1 / 93.3 / 93.9%). Our method does not change **how often** the model is right. It changes **why**.

YOLO + C: YOLO Proposes, C Decides



- **YOLOv8s** (COCO-pretrained, fine-tuned on the boxes): its best box is on the fracture for **79%** of the X-rays, better than any explanation (67%).
- Each of YOLO's 10 boxes is re-scored with C:
 $p \cdot c^{1-\alpha} \cdot h^\alpha$, $\alpha = 0.5$ chosen on val.
- C and YOLO **agree** $\Rightarrow$ decide (76% of the X-rays, **96.6%** right); they disagree $\Rightarrow$ radiologist review (24%).
- Fractures missed: **5.8%** (6 of 104).

Test: YOLO vs YOLO + C



Top: YOLO's best box is off the fracture; C's Grad-CAM lifts the right one: miss → **hit**.

Bottom: both miss: C can only choose among YOLO's boxes.

Recap on all test X-rays

- **Placing the box:** no gain: 76% vs 79% (not significant). YOLO's own ranking is already good.
- **Fewer false boxes:** yes: AP@0.5 on all X-rays **40.7** → **46.9**, boxes on healthy X-rays fall to the bottom.
- **Pipeline:** when C and YOLO agree it decides 76% of the X-rays, **96.6%** right; the rest goes to a radiologist.

⇒ For *where*, use YOLO; for *whether*, use C.

Limitations and Lessons

Limitations

- One dataset; one seed per run (two full runs of A, B, C).
- 104 fractured test X-rays: 4 points = 4 X-rays. Deletion test on 10 X-rays.
- Loss weights, thresholds, α set once, not tuned.
- The pointing game is strict: one point, a tiny box.
- C is trained with the blur it is tested with (controls: random patch, five untrained maps).
- Casts and metal plates still fool every model.
- The built-in detector stays far behind YOLO (AP 22–31% vs 53%).

Lessons

- **Accuracy is not enough:** 80.9% (CNN) can mean “finds 1 fracture in 6”.
- **Pretraining and resolution** beat hand-crafted invariance.
- A faithful map can still show a **shortcut**; the model must be **taught** to look.
- **Score the layer you train**, and test the decision, not only the map.
- A **specialist** stays best at its job: use YOLO for boxes, our model as a second opinion.

Conclusion

- **CNN vs ScatNet**: statistically tied and close to “never fractured”; a pretrained ResNet18 reaches **91.5%**.
- **Six XAI methods**: the most faithful depends on the model (Grad-CAM for the CNN, LIME for ResNet18); our Occlusion equals Captum; but the maps point at the fracture at most **23%** of the time.
- **Our joint model C** (classifier + detector + explanation losses, 448 px):
 - keeps the accuracy (**93.0%**);
 - Grad-CAM hits the fracture **67%** of the time instead of 35% (A); the other five improve too, significantly only Guided Backprop;
 - blurring the fracture now changes the answer (84% → **2.5%**).
- **YOLO** remains the best box finder (79%); our model helps it by removing false boxes (AP on all X-rays 40.7 → 46.9), and together they decide 76% of the X-rays with **97%** accuracy.

Thank you for your attention!

Code, results and figures: github.com/silvergjecha22/bone-fracture-detection